

ARCHITECTURE WHITE PAPER

Nexus Observability

Real-Time Workflow Observability & System Health
Platform

This document provides a comprehensive technical reference for the Nexus Observability platform — covering its purpose, system architecture, core feature set, API design, data flow, and engineering decisions. Intended for engineering teams, solution architects, and technical stakeholders.

DOCUMENT VERSION

1.0 — July 2026

DEPLOYMENT STATUS

Production

CORE STACK

React 19 · Supabase · Vercel

CLASSIFICATION

Confidential — Internal



NAVIGATION

Table of Contents

1.	What Is This Platform?
2.	Live URLs
3.	Technology Stack
4.	Application Architecture
	↳ Figure 1: High-Level Architecture & Data Flow
	↳ Figure 2: Low-Level Flow Architecture
5.	Key Features — Detailed Breakdown
6.	API Routes
7.	How to Connect a Backend Service
8.	Data Flow Diagram
9.	Authentication
10.	Offline / Degraded Mode
11.	Custom Hooks
12.	Key Design Decisions
13.	Repository Structure
14.	Future Feature Ideas

Nexus Observability — Complete Project Context Document

A living reference document that explains the purpose, architecture, features, and technical decisions behind the Nexus Observability platform.

1. What Is This Platform?

Nexus Observability is a real-time workflow observability platform that lets engineering teams define, visualize, and monitor the health of complex multi-service systems. Think of it as a combination of a **visual architecture builder** and a **live system health dashboard** — all in one.

The core problem it solves: when something goes wrong in a distributed system (e.g., a payment pipeline, a stock processing chain, or an employee onboarding workflow), engineers need to quickly answer three questions:

- **Where exactly did it fail?** (Which component?)
- **What is downstream affected?** (What else will break because of this?)
- **Why did it fail?** (What do the logs, events, and traces say?)

Nexus Observability answers all three — visually, in real time.

2. Live URLs

ENVIRONMENT	URL
Production	<code>https://stock-observability-ui.vercel.app</code>
Inspect / Deployments	<code>https://vercel.com/prathamesh-s-projects6/stock-observability-ui</code>

3. Technology Stack

Frontend

TECHNOLOGY	VERSION	PURPOSE
React	19.x	Core UI framework
Vite	8.x	Build tool and development server
ReactFlow	11.x	Interactive node graph canvas (builder + dashboard)
Recharts	3.x	Chart components for metrics and analytics
react-router-dom	7.x	URL-based client-side routing and navigation
react-icons	5.x	Icon library (Feather icons used throughout)
Vanilla CSS	—	All styling; no Tailwind classes used at runtime

Backend / Infrastructure

TECHNOLOGY	PURPOSE
Supabase	Postgres database + Realtime WebSocket subscriptions + Auth
Vercel Serverless Functions	All API routes (Node.js, file-based routing)
Vercel	Hosting, CI/CD, edge CDN

Database (Supabase / Postgres)

Key tables:

- `workflows` — stores workflow definitions (name, components JSON, nodes JSON, edges JSON, common link)
- `events` — raw observability events emitted by components (status, duration, entity id, etc.)
- `metrics` — time-series performance metrics (latency, throughput, CPU) per component

- `logs` — structured application log lines per component
 - `alerts` — triggered alert records per workflow
 - `alert_rules` — user-configured alert thresholds per component
-

4. Application Architecture

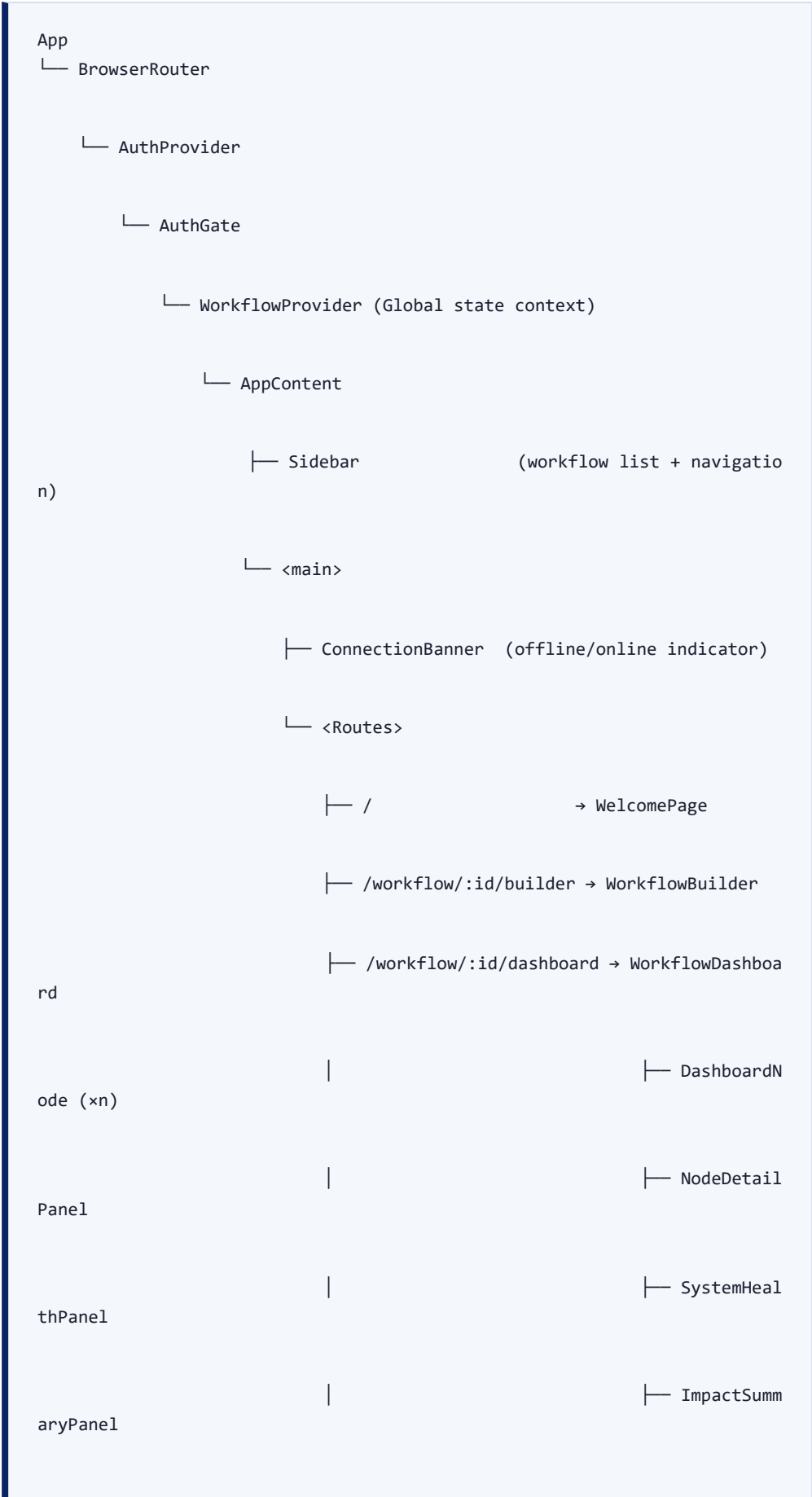
URL Structure (React Router)

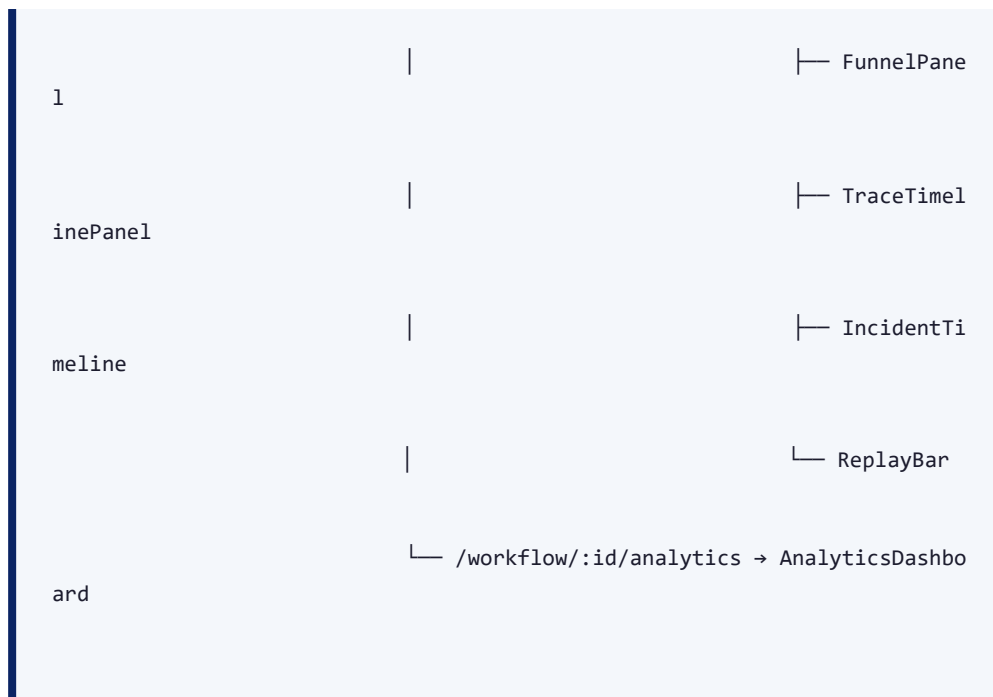
The app is a **Single Page Application (SPA)** using `BrowserRouter`. Every view is a distinct URL, enabling browser back/forward history, refreshes, and shareable deep links.

<code>/</code>	→ Welcome screen (list of all workflows)
<code>/workflow/:id/builder</code>	→ Workflow Builder (visual layout editor)
<code>/workflow/:id/dashboard</code>	→ Live Monitoring Dashboard
<code>/workflow/:id/analytics</code>	→ Historical Analytics view

Vercel is configured with a catch-all rewrite rule so that any non-API URL serves `index.html`, allowing React Router to handle navigation client-side.

High-Level Component Tree





State Management

Global state is managed via a **React Context** (`WorkflowContext`). It is responsible for:

- ▶ Holding the full list of `workflows` loaded from the API
- ▶ Providing CRUD operations: `createWorkflow`, `deleteWorkflow`, `duplicateWorkflow`, `renameWorkflow`, `saveWorkflow`
- ▶ Exposing per-component operations: `addComponent`, `updateComponent`, `removeComponent`
- ▶ Managing `editingWorkflowId` (for the builder)
- ▶ Replay Mode state: `replayMode`, `replayTimestamp`, `enterReplay`, `exitReplay`
- ▶ WebSocket health state: `wsStatus`, `wsAttemptCount`, `triggerReconnect`

Important: Navigation state (active view, active workflow) is no longer in context. It lives in the URL via `useParams()` and `useNavigate()`.

5. Key Features — Detailed Breakdown

5.1 Workflow Builder

File: `WorkflowBuilder.jsx`

The builder is a drag-and-drop canvas (powered by ReactFlow) where users define their architecture:

- **Add Components** — A form modal captures the component name, manager/owner, and SLA (Service Level Agreement) target.
- **Draw Connections** — Drag from the blue connection handles on any node to another node to create a directed edge.
- **Edge Direction** — Clicking an edge opens a context menu to toggle between **one-way** ($\rightarrow$) and **two-way** ($\rightleftarrows$) data flow.
- **Auto-Discover** — Attempts to auto-detect topology from connected infrastructure.
- **Auto-Layout** — Automatically positions all nodes in a clean left-to-right hierarchical arrangement.
- **Common Link Field** — A shared field (e.g., "Employee Email" or "Order ID") that links records across all components in the workflow, enabling entity-level tracing.
- **Save & Monitor** — Persists the workflow to Supabase via the API and navigates to the live dashboard.

5.2 Live Monitoring Dashboard

File: `WorkflowDashboard.jsx`

The dashboard renders the same workflow graph but in **read-only, live-monitoring mode**. Each node displays live health data.

Real-time data flow:

- On mount, the dashboard subscribes to Supabase Realtime channels for `events`, `metrics`, `logs`, and `alerts` tables, filtered by the active `workflow_id`.
- When a new record is inserted (by a backend service calling the webhook), the dashboard updates instantly without polling.
- If the WebSocket drops, the platform detects this, shows a "Reconnecting" banner, and attempts automatic recovery.

Fallback: If Supabase is not configured, the dashboard runs entirely on local state with simulated events.

5.3 Node Detail Panel

File: `NodeDetailPanel.jsx`

Clicking any component node on the dashboard opens a slide-in panel with multiple tabs:

TAB	CONTENTS
Overview	Live status, latency, throughput, CPU usage, SLA compliance bar
Logs	Scrollable real-time log stream for this component
Metrics	Recharts line graphs for latency, throughput, and CPU over time
Alert Rules	Create, view, and delete alert thresholds (e.g., "alert if latency > 500ms")
Integration	Auto-generated code snippet to connect a backend service to this component's webhook
Profiling	Flamegraph visualization for performance profiling traces

5.4 Impact Map (Dependency Blast Radius)

Files: `WorkflowDashboard.jsx`, `ImpactSummaryPanel.jsx`

When you activate **Impact Mode** on a failing component, a **Breadth-First Search (BFS)** algorithm traverses the workflow's edge graph to compute every downstream node that depends on the source component — directly or transitively.

The affected nodes are highlighted, unaffected nodes are dimmed, and the `ImpactSummaryPanel` lists the specific downstream services at risk. This gives responders an immediate answer to: "If this service goes down, what else breaks?"

5.5 Entity Trace (End-to-End Journey Tracking)

The **Track** feature lets you search by a `common link` value (e.g., a specific order ID `ORD-9821` or an employee email `alice@company.com`). The platform then queries every component's event log for that specific entity and reconstructs its full journey through the workflow — showing which stages it passed, which it failed, and where it got stuck.

This is powered by:

- ▶ `GET /api/workflows/:id/trace/:entityId` — backend API route

- `fetchEntityTrace()` — frontend API helper
- `TraceTimelinePanel.jsx` — visual timeline rendering

5.6 Transaction Funnel

File: `FunnelPanel.jsx`

The Funnel shows the **conversion rate** of entities as they move through each stage of the workflow. It identifies exactly where in the pipeline the highest drop-off is occurring — like a sales funnel but for system reliability.

5.7 Critical Path Highlighting

The **Critical Path** is the chain of components with the highest cumulative latency — the bottleneck chain that is limiting overall system throughput. When activated, the edges along the critical path are visually distinguished on the canvas.

5.8 Historical Analytics

File: `AnalyticsDashboard.jsx`

Aggregates historical event data across all components for a workflow and presents summary statistics:

- Total events processed
- Overall success vs. failure rates
- Per-component breakdown
- Average processing times

Powered by `GET /api/workflows/:id/analytics`.

5.9 Replay Mode

File: `ReplayBar.jsx`

A time-slider bar (fixed at the bottom of the dashboard) that lets you **rewind the system state to any point in the past 24 hours**. When active, all components display the status they had at the chosen timestamp rather than live data. This is critical for post-incident reviews — you can recreate the exact system state at the moment of a past outage.

5.10 System Health Panel

File: `SystemHealthPanel.jsx`

A collapsible panel showing the aggregate health score of the entire workflow — a high-level "traffic light" summary combining all component statuses.

6. API Routes (Vercel Serverless Functions)

All API functions live in the `api/` directory. They connect to Supabase using the **Service Role Key** (bypasses Row Level Security) and return JSON.

Workflow CRUD

METHOD	ROUTE	DESCRIPTION
GET	<code>/api/workflows</code>	List all workflows
POST	<code>/api/workflows</code>	Create a new workflow
GET	<code>/api/workflows/:id</code>	Get a single workflow
PUT	<code>/api/workflows/:id</code>	Update a workflow (name, components, nodes, edges)
DELETE	<code>/api/workflows/:id</code>	Delete a workflow
POST	<code>/api/workflows/:id/duplicate</code>	Clone a workflow with a new ID

Workflow Intelligence

METHOD	ROUTE	DESCRIPTION
GET	/api/workflows/:id/events	Recent events for a workflow (last 100)
GET	/api/workflows/:id/trace/:entityId	All events for one entity across all components
GET	/api/workflows/:id/analytics	Aggregate statistics
GET	/api/workflows/:id/alert-rules	List alert rules for a component
POST	/api/workflows/:id/alert-rules	Create an alert rule
DELETE	/api/workflows/:id/alert-rules	Delete an alert rule
GET/POST	/api/workflows/:id/discover	Auto-discover topology
GET	/api/workflows/:id/profiles	Performance profiling data

Event Ingestion (Webhook)

METHOD	ROUTE	DESCRIPTION
POST	/api/v1/events	Main webhook endpoint. External services POST events here. The handler auto-detects which workflow the <code>component_id</code> belongs to.
POST	/api/v1/[...otel]	OpenTelemetry-compatible trace ingestion endpoint

Other

METHOD	ROUTE	DESCRIPTION
GET	/api/cron/archive	Scheduled cron job (runs daily at midnight) to archive old events

7. How to Connect a Backend Service

Any external system can push observability events to the platform by making a single HTTP POST:

```
POST https://stock-observability-ui.vercel.app/api/v1/events
Content-Type: application/json

{

  "component_id": "comp_your_component_id",

  "entity_id": "order-12345",

  "status": "success",

  "duration_ms": 142,

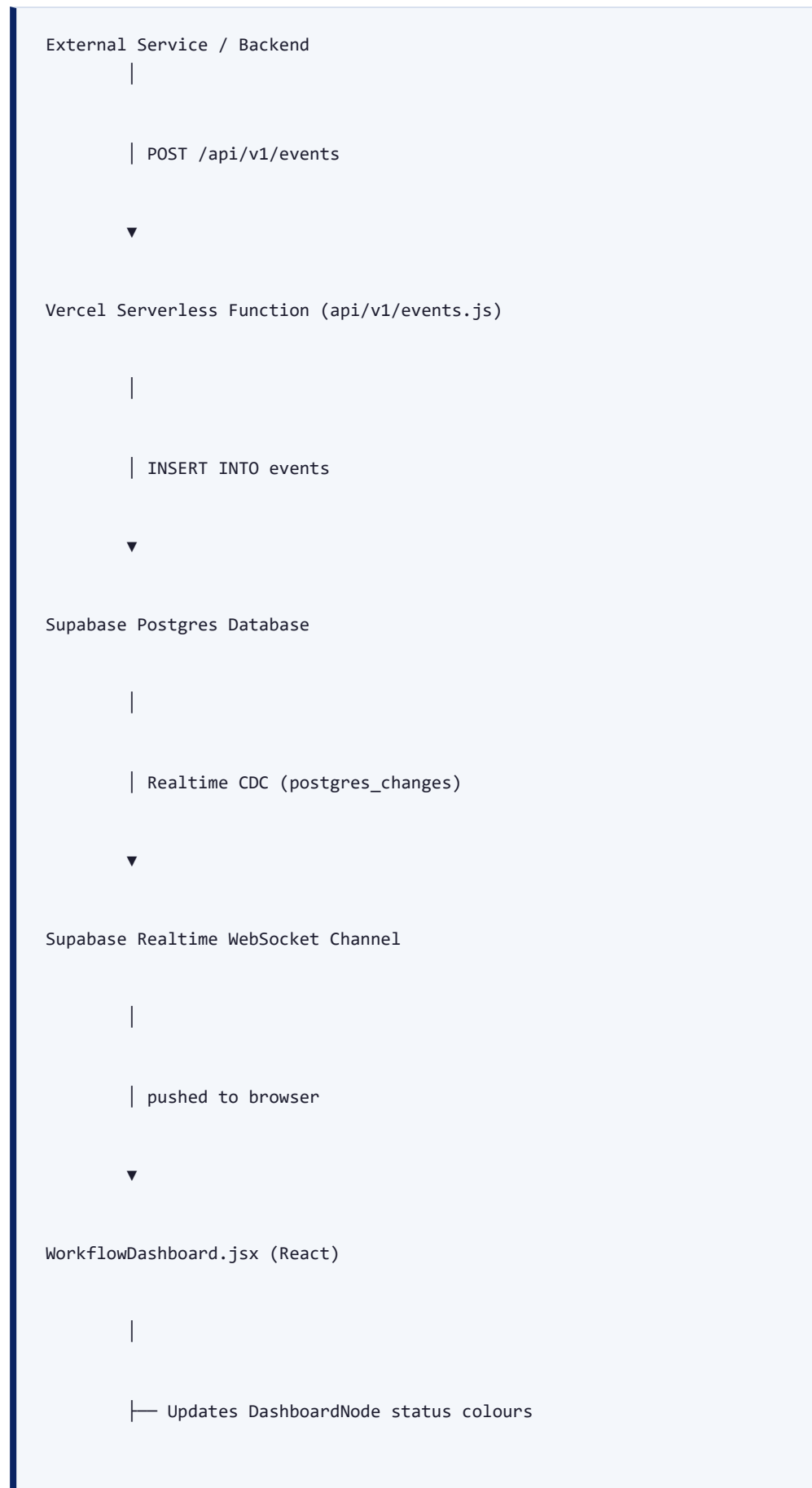
  "message": "Order processed successfully",

  "metadata": { "region": "us-east-1" }

}
```

The platform automatically associates the event to the correct workflow by looking up the `component_id` in the `workflows` table's `components` JSONB array.

8. Data Flow Diagram



```
└─ Appends to IncidentTimeline

└─ Updates MetricsPanel charts

└─ Triggers alert checks
```

9. Authentication

File: `src/auth/AuthProvider.jsx`

Authentication is handled by **Supabase Auth**. On app load, `AuthProvider` calls `supabase.auth.getSession()` and listens for `onAuthStateChange`. The `AuthGate` component wraps the entire app — only authenticated sessions can access the dashboard. Unauthenticated users are shown a login screen.

10. Offline / Degraded Mode

The platform is designed to be resilient when Supabase is unavailable:

- If environment variables `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY` are missing, the Supabase client is `null`.
- The `isSupabaseEnabled` flag (exported from `supabase.js`) guards all real-time subscriptions.
- The `WorkflowContext` falls back to **localStorage** for workflow persistence if the API is unreachable.
- A `ConnectionBanner` component notifies the user when they are in offline mode.

11. Custom Hooks

HOOK	FILE	PURPOSE
<code>useRealtimeStatus</code>	<code>hooks/useRealtimeStatus.js</code>	Exposes the current WebSocket connection health (connected / reconnecting / failed)
<code>useReplayData</code>	<code>hooks/useReplayData.js</code>	When Replay Mode is active, fetches the historical snapshot of events for a given workflow at the replay timestamp

12. Key Design Decisions

DECISION	RATIONALE
ReactFlow for the canvas	Provides the interactive node-graph primitives (drag, zoom, pan, edge drawing) out of the box, without needing to implement SVG rendering from scratch
Supabase Realtime instead of polling	WebSocket CDC push means latency from event ingestion to dashboard update is ~50-200ms vs. ~5s for polling
URL-based routing instead of state-based	Enables browser history, refreshing without losing context, and shareable deep links to specific workflow dashboards
Vercel Serverless Functions	Eliminates the need for a dedicated backend server; scales to zero when idle; integrates natively with Vercel deployment pipeline
Optimistic UI updates	All CRUD operations (create, rename, delete workflow) update the local React state immediately before the API call completes, making the UI feel instant
localStorage fallback	Guarantees the app remains usable even when the backend API is unreachable
BFS for Impact Map	Breadth-First Search is the correct algorithm for "all reachable nodes from a source" in a directed graph. It correctly handles multi-hop dependencies and cycles.



13. Repository Structure

```

stock-observability-ui/
├─ api/                                # Vercel Serverless API routes
|
|   ├─ v1/
|   |
|   |   ├─ events.js                  # Main webhook ingestion endpoint
|   |   └─ [...otel].js              # OpenTelemetry trace endpoint
|   └─ workflows/
|       ├─ index.js                  # GET all / POST create workflow
|       ├─ [id].js                   # GET / PUT / DELETE single workflow
|       └─ [id]/
|           ├─ events.js              # Recent events
|           ├─ analytics.js           # Aggregate stats
|           ├─ alert-rules.js         # Alert rule CRUD
|           ├─ discover.js            # Topology auto-discovery
|           └─ profiles.js            # Performance profiles
|       └─ trace/                    # Entity trace endpoint
|   └─ cron/
|       └─ archive.js                # Daily cleanup job
|
└─ src/

```

```
| └─ App.jsx                # Root component; routing config

| └─ main.jsx              # React entry point

| └─ index.css             # All global styles and design tokens

| |

| └─ auth/

| |   └─ AuthProvider.jsx  # Supabase Auth session management

| |

| └─ context/

| |   └─ WorkflowContext.jsx # Global state (workflows, CRUD, replay)

| |

| |

| └─ hooks/

| |   └─ useRealtimeStatus.js # WebSocket health hook

| |   └─ useReplayData.js    # Replay mode data fetcher

| |

| └─ lib/

| |   └─ supabase.js        # Supabase client singleton

| |   └─ api.js             # Frontend API helper functions

| |   └─ realtimeStatus.js  # WebSocket event bus

| |
```

```
| └─ components/

|   └─ WorkflowBuilder.jsx  # Visual layout editor

|   └─ WorkflowDashboard.jsx # Live monitoring dashboard

|   └─ AnalyticsDashboard.jsx# Historical analytics

|   └─ Sidebar.jsx          # Left navigation panel

|   └─ NodeDetailPanel.jsx  # Slide-in component detail (tabs)

|   └─ DashboardNode.jsx    # ReactFlow node (dashboard mode)

|   └─ BuilderNode.jsx      # ReactFlow node (builder mode)

|   └─ MetricsPanel.jsx     # Recharts metrics charts

|   └─ SystemHealthPanel.jsx # Overall health summary

|   └─ ImpactSummaryPanel.jsx# Blast radius summary list

|   └─ FunnelPanel.jsx      # Conversion funnel chart

|   └─ TraceTimelinePanel.jsx# Entity journey timeline

|   └─ IncidentTimeline.jsx # Live incident event log

|   └─ ReplayBar.jsx        # Time-travel replay scrubber

|   └─ Flamegraph.jsx       # CPU profiling flamegraph

|   └─ StaleNodeOverlay.jsx # "Data stale" indicator overlay

|   └─ ConnectionBanner.jsx # Offline/online status banner

|   └─ ComponentForm.jsx    # Add/edit component modal form

|   └─ ErrorBoundary.jsx    # React error boundary wrapper
```

```
|  
  
└─ vercel.json # Vercel config (functions, rewrites, c  
ron)
```

14. Future Feature Ideas

The following have been discussed as potential next steps:

- **Health Status History / Uptime Tracking** — Record historical uptime percentages per component and display them as availability SLAs over rolling 7/30/90-day windows.
- **Multi-Tenant Support / RBAC** — Support multiple organizations with role-based access control (Viewer, Editor, Admin) per workflow.
- **Automated SLA Breach Reports** — Automatically generate and email PDF or Slack reports when a component's SLA is breached beyond a configured threshold.
- **UI De-cluttering** — Collapsible side panels, a cleaner tab hierarchy, and better responsive layout for smaller screens.

Last updated: July 2026